

Self-supervised Object-Centric Learning for Videos

Paper: <https://arxiv.org/abs/2310.06907>

Code: <https://github.com/gorkaydemir/SOLV>

Report by: Margareth - 02522025

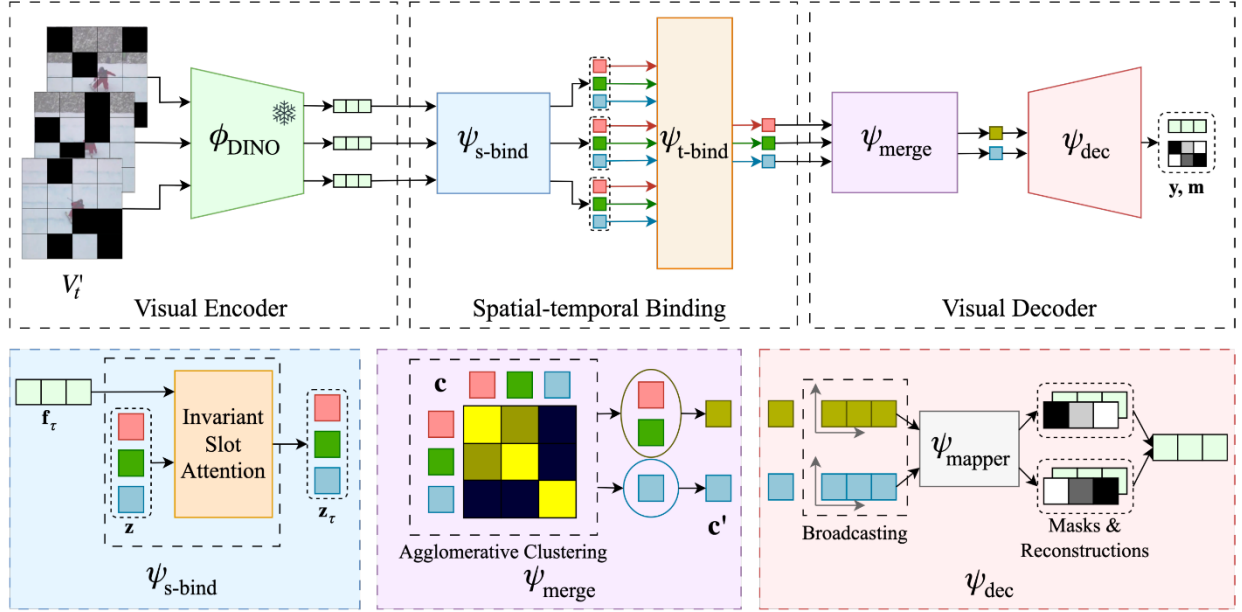
Paper Summary

- The authors present an unsupervised training method for segmenting multiple objects in real-world sequences which is found in video processing
- The object-centric framework spatially binds objects to slots on each frame and relates these slots across frames
- A masking strategy is proposed in which a significant portion of tokens in the feature space is dropped for efficiency and regularization
- They also address over-clustering by merging slots based on similarity
- Successfully segmented multiple instances of complex and high-variety classes in YouTube videos

1 Introduction

Goal of the study is to train a vision system to discover, track, and segment objects to abstract visual information (object-centric visual representation learning). Compared to image data, videos need additional modalities to indicate the assignment of pixels which incur computation overhead and error accumulation. This paper circumvents this issue by adopting spatial-temporal slot attentions to group visual regions with similar property without relying on additional signal. Following object-centric learning approach, the algorithm use masked feature reconstruction and slot merging which has the benefit of serving as information bottleneck to force the model to learn high-level semantic structures from only partial observations and it also reduces memory constraints which improves computational efficiency.

2 Architecture



- Visual Encoder

Divide each image into regular non-overlapping patches followed by token extraction. The visual encoder consists of token drop and feature extraction.

- Spatial-temporal Binding

After extracting visual features of each frame, the image is spatially grouped into **slots**. Each slots specifying a semantic object (discovering objects within a single image). Then, temporal bindings are established between slots with a Transformer (associate objects within video clips)

Spatial binding is applied to each frame independently using invariant slot attention with shared initialization. Given features of after token drop at a time step as input, a set of initialization vectors are learned to translate and scale the input position encoding for each slot separately with slot vectors, scale vectors, position vectors, and one absolute positional embedding grid. Patches corresponding to the dropped tokens is masked out to obtain the absolute positional embedding of each frame.

- Visual Decoder

Due to the set number of slots used, over-clustering become a problem in videos application due to variation in the number of objects inside a frame. Hence, through Agglomerative Clustering they do slot merging to overcome this problem. And to reconstruct back the video features, they have to do the reverse process of slot decoding (resembling masked auto-encoder in feature space)

3 Analysis of the training code:

Function definition:

- Train_epoch
 - Runs one epoch of training. It iterates through the dataloader, uses visual encoder to get output features and proceed to drop some of the features
 - The function then reconstruct based on the dropped features and calculate the MSE loss of the model
 - Optimizer gradient is reset by optimizer.zero_grad() prior to calculating the new gradient in the optimization step afterward
 - Return the mean loss and total iteration
- Val_epoch
 - Runs under the wrapper torch.no_grad()
 - First split input into chunks and extract DINO features by using the visual encoder and concatenating it into a lists with the token indices. It creates a sliding windows of $2N+1$ for each target frame and processes them in batches, then creates reconstruction by calling the model and giving the flattened features, input masks, and token indices. The resulting reconstruction mask, slots, and slot nums are reshaped, interpolated, and amplified to create predictions. Resulting predictions update the evaluator and this function return mIoU and FG-ARI
- Main_worker
 - Starts by preparing dataloader to load data, creating instance of Visual_Encoder and initializing the SOLV model
 - Initializes Adam optimizer for optimizing the parameter for the model and get scheduler from the optimizer and train dataloader
 - For validation, it runs the val_epoch function above

For training, it runs through a training loop over epochs which loads the input data through the dataloader, in each epoch the model parameter is updated and runs train_epoch function above, saving the model state after each train_epoch calls as checkpoint. Continued by validation run using val_epoch and finally printing the total training time required and closed by destroying the process group

Train.py

```
import os
import time
import math
import datetime

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.backends.cudnn as cudnn
import torch.distributed as dist
import torch.multiprocessing as mp

from tqdm import tqdm

from models.model import SOLV, Visual_Encoder
from read_args import get_args, print_args
import utils

def train_epoch(args, vis_encoder, model, optimizer, scheduler, train_dataloader, total_iter, writer):
    total_loss = 0.0
    vis_encoder.eval()
    model.train()

    loader = tqdm(train_dataloader, disable=(args.gpu != 0))

    for i, (frames, masks) in enumerate(loader):
        frames = frames.cuda(non_blocking=True)          # (B, F, 3, H, W)
        masks = masks.cuda(non_blocking=True)           # (B, F)

        B = frames.shape[0]
        with torch.cuda.amp.autocast(True):
            output_features, _ = vis_encoder(frames[:, [args.N]], get_gt=True)
            dropped_features, token_indices = vis_encoder(frames, get_gt=False)

            assert output_features.isnan().any() == False, f"{torch.sum(output_features.isnan())} items  
are NaN"
            assert dropped_features.isnan().any() == False, f"{torch.sum(dropped_features.isnan())} items  
are NaN"

            output_features = output_features.to(torch.float32)
            dropped_features = dropped_features.to(torch.float32)

            reconstruction = model(dropped_features, masks, token_indices)
            loss = F.mse_loss(reconstruction["rec"], output_features).mean()

            total_loss += loss.item()

            optimizer.zero_grad(set_to_none=True)

            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()

            scheduler.step()

            if args.gpu == 0:
                lr = optimizer.state_dict()["param_groups"][0]["lr"]
                mean_loss = total_loss / (i + 1)
                loader.set_description(f"lr: {lr:.6f} | loss: {mean_loss:.5f}")

                writer.add_scalar("batch/loss", loss.item(), total_iter)

            total_iter += 1

    mean_loss = total_loss / (i + 1)
    return mean_loss, total_iter
```

```

@torch.no_grad()
def val_epoch(args, vis_encoder, model, val_dataloader, evaluator, writer, epoch):
    vis_encoder.eval()
    model.eval()
    val_loader = tqdm(val_dataloader)

    bs = args.batch_size // args.gpus

    for i, (model_input, input_masks, gt_masks) in enumerate(val_loader):

        model_input = model_input.cuda(non_blocking=True)           # (1, #frames + 2N, 3, H, W)
        input_masks = input_masks.cuda(non_blocking=True)          # (1, #frames + 2N)
        gt_masks = gt_masks.cuda(non_blocking=True)                # (1, #frames, H_t, W_t)

        H_t, W_t = gt_masks.shape[-2:]
        frame_num = gt_masks.shape[1]

        H, W = args.resize_to
        turn_number = model_input.shape[1] // bs
        if model_input.shape[1] % bs != 0:
            turn_number += 1

        # === DINO feature extraction ===
        all_dino_features = []
        all_token_indices = []
        for j in range(turn_number):
            s = j * bs
            e = (j + 1) * bs
            with torch.cuda.amp.autocast(True):
                features, token_indices = vis_encoder(model_input[:, s:e], get_gt=True)  # (bs,
token_num, 768), (bs, token_num)
                assert features.isnan().any() == False, f"{torch.sum(features.isnan())} items are NaN"

            all_dino_features.append(features.to(torch.float32))
            all_token_indices.append(token_indices)

        all_dino_features = torch.cat(all_dino_features, dim=0)      # (#frames + 2N,
token_num, 768)
        all_token_indices = torch.cat(all_token_indices, dim=0)    # (#frames + 2N,
token_num)

        all_model_inputs = []
        all_model_tokens = []
        all_masks_input = []
        for i in range(frame_num):
            indices = list(range(i, i + (2 * args.N + 1)))
            all_model_inputs.append(all_dino_features[indices].unsqueeze(dim=0))  # (1, 2N + 1,
token_num, 768)
            all_model_tokens.append(all_token_indices[indices].unsqueeze(dim=0))  # (1, 2N + 1,
token_num)
            all_masks_input.append(input_masks[:, indices])              # (1, 2N + 1)

        all_model_inputs = torch.cat(all_model_inputs, dim=0)      # (#frames, 2N + 1,
token_num, 768)
        all_model_tokens = torch.cat(all_model_tokens, dim=0)      # (#frames, 2N + 1,
token_num)
        all_masks_input = torch.cat(all_masks_input, dim=0)        # (#frames, 2N + 1)
        # === === ===

        turn_number = frame_num // bs
        if frame_num % bs != 0:
            turn_number += 1

        out_masks = []
        all_slots = []
        all_slot_nums = []
        for j in range(turn_number):
            s = j * bs

```

```

        e = (j + 1) * bs

        # === Input features ===
        features = all_model_inputs[s:e]                    # (bs, 2N + 1, token_num, 768)
        features = torch.flatten(features, 0, 1)            # (bs * (2N + 1), token_num, 768)

        # === Token indices ===
        token_indices = all_model_tokens[s:e]               # (bs, 2N + 1, token_num)
        token_indices = torch.flatten(token_indices, 0, 1)  # (bs * (2N + 1), token_num)

        # === Attention masks ===
        input_masks_j = all_masks_input[s:e]

        reconstruction = model(features, input_masks_j, token_indices)

        masks = reconstruction["mask"]                      # (bs, S, token)
        slots = reconstruction["slots"]                     # (bs, S, D_slot)
        slot_nums = reconstruction["slot_nums"]             # (bs)
        for l in range(slot_nums.shape[0]):
            slot_num = slot_nums[l]
            slots_l = slots[l, :slot_num]                   # (S', D_slot)
            all_slots.append(slots_l)

        out_masks.append(masks)
        all_slot_nums.append(slot_nums)

    all_slots = torch.cat(all_slots, dim=0)                 # (#slots, D_slot)
    all_slot_nums = torch.cat(all_slot_nums, dim=0)         # (#frames)
    masks = torch.cat(out_masks, dim=0)                    # (#frames, S,
token)

    S = masks.shape[1]

    masks = masks.view(-1, S, H // args.patch_size, W // args.patch_size)  # (#frames, S, H
// 8, W // 8)
    predictions = F.interpolate(masks, size=(H_t, W_t), mode="bilinear")      # (#frames, S,
H_t, W_t)
    predictions = torch.argmax(predictions, dim=1)                          # (#frames, H_t,
W_t)

    if args.merge_slots:
        predictions = utils.bipartiate_match_video(all_slots, all_slot_nums, predictions)

    # === Instance Segmentation Evaluation ===
    miou = evaluator.update(predictions, gt_masks[0])
    loss_desc = f"mIoU: {miou * 100:.3f}"

    # === Logger ===
    val_loader.set_description(loss_desc)
    # === === ===

    # === Evaluation Results ===
    miou, fg_ari = evaluator.get_results()

    # === Logger ===
    print("\n=== Results ===")
    print(f"\tmIoU: {miou * 100:.3f}")
    print(f"\tFG-ARI: {fg_ari * 100:.3f}\n")

    # === Tensorboard ===
    writer.add_scalar("multi_object/mIoU", miou, epoch)
    writer.add_scalar("multi_object/FG-ARI", fg_ari, epoch)

    return miou, fg_ari

def main_worker(args):
    utils.init_distributed_mode(args)
    utils.fix_random_seeds(args.seed)
    cudnn.benchmark = True

```

```

print_args(args)

# === Dataloaders ===
train_dataloader, val_dataloader = utils.get_dataloaders(args)

# === Model ===
vis_encoder = Visual_Encoder(args).cuda()
model = SOLV(args).cuda()

model = nn.parallel.DistributedDataParallel(model, device_ids=[args.gpu], find_unused_parameters=True)

# === Training Items ===
optimizer = torch.optim.Adam(utils.get_params_groups(model), lr=args.learning_rate)

scheduler = utils.get_scheduler(args, optimizer, train_dataloader)

# === Misc ===
evaluator = utils.Evaluator() if args.gpu == 0 else None
writer = utils.get_writer(args) if args.gpu == 0 else None

print(f"Loss, optimizer and schedulers ready.")

# === Load from checkpoint ===
to_restore = {"epoch": 0}
if args.use_checkpoint:
    utils.restart_from_checkpoint(args,
                                run_variables=to_restore,
                                model=model,
                                optimizer=optimizer,
                                scheduler=scheduler)

start_epoch = to_restore["epoch"]

start_time = time.time()

dist.barrier()

if args.validate:

    print("Starting SOLV evaluation!")
    model.module.merge.cluster_drop_p = 0
    if args.gpu == 0:
        val_epoch(args, vis_encoder, model, val_dataloader, evaluator, writer, 0)

    dist.barrier()
    total_time = time.time() - start_time
    total_time_str = str(datetime.timedelta(seconds=int(total_time)))
    print('Validation time {}'.format(total_time_str))
    dist.destroy_process_group()
    return

print("Starting SOLV training!")

total_iter = 0
best_miou = -1
for epoch in range(start_epoch, args.num_epochs):
    train_dataloader.sampler.set_epoch(epoch)

    print(f"===== [Epoch {epoch}] =====")

    model.module.merge.cluster_drop_p = 1 - (math.log(epoch + 1) / math.log(args.num_epochs))
    mean_loss, total_iter = train_epoch(args, vis_encoder, model, optimizer, scheduler,
train_dataloader, total_iter, writer)

# === Save Checkpoint ===
save_dict = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),

```

```

        "scheduler": scheduler.state_dict(),
        "epoch": epoch + 1,
        "args": args,
    }

    utils.save_on_master(save_dict, os.path.join(args.model_save_path, "checkpoint.pt"))
    if epoch % 5 == 4:
        utils.save_on_master(save_dict, os.path.join(args.model_save_path,
f"checkpoint_epoch_{epoch}.pt"))

    # === Validate ===
    if args.gpu == 0:
        if (epoch == 0) or ((epoch + 1) % args.validation_epoch == 0):
            model.module.merge.cluster_drop_p = 0
            miou, _ = val_epoch(args, vis_encoder, model, val_dataloader, evaluator, writer, epoch)
            if miou > best_miou:
                best_miou = miou
                utils.save_on_master(save_dict, os.path.join(args.model_save_path,
f"best_checkpoint.pt"))

        # === Log ===
        writer.add_scalar("epoch/train-loss", mean_loss, epoch)
        writer.flush()
        writer.close()

    dist.barrier()

    print("==== ===== ===== ===== =====")

    total_time = time.time() - start_time
    total_time_str = str(datetime.timedelta(seconds=int(total_time)))
    print(f"Training time {total_time_str}")
    dist.destroy_process_group()

if __name__ == '__main__':
    args = get_args()
    main_worker(args)

```